Architectural Foundations of Relational Database Integrity: Key Taxonomy, Physical Storage Indexing, Referential Constraints, and Mutation Mechanics in SQL Systems

Abstract

In relational database management systems (RDBMS), structural integrity and storage efficiency depend fundamentally on key constraints and referential enforcement protocols. While classical database literature often obscures implementation realities behind abstract relational mathematics, production engineering requires a concrete understanding of physical storage layout, B+ Tree index clustering, memory-cache locality, and foreign key state transitions. This paper presents an exhaustive architectural analysis of relational keys, constraint enforcement, and SQL CRUD operations based on modern database engines such as MySQL InnoDB. We examine the taxonomy of database keys—distinguishing candidate keys, surrogate primary keys, natural composite keys, and foreign keys. We analyze the physical mechanics of primary key selection, demonstrating how data types, byte footprints, lexicographic versus integer sorting, and immutability directly govern B+ Tree page fan-out and I/O efficiency. Furthermore, we dissect foreign key constraints and referential integrity, tracing the exact consequences of parent record mutations. We formalize the operational mechanics of ON UPDATE and ON DELETE actions—CASCADE, SET NULL, and RESTRICT/NO ACTION—contrasting high-impact architectural patterns across enterprise domains, from unified single sign-on user lifecycles (Google Account cascade deletion) to regulatory audit trail preservation (Amazon order history decoupling). Finally, we demonstrate the operational lifecycle of database schemas through production-grade SQL DDL and DML workflows, providing concrete patterns for resilient schema design in high-throughput transactional systems.

Index Terms

Relational Databases, SQL CRUD Mechanics, Candidate Keys, Primary Keys, Composite Keys, Foreign Keys, Referential Integrity, ON UPDATE CASCADE, ON DELETE SET NULL, Clustered Index, B+ Tree, InnoDB Storage Engine, Sakila Database

1 Introduction and Relational Architecture Foundations

Modern software engineering systems depend fundamentally on Relational Database Management Systems (RDBMS) to guarantee data persistence, transaction safety, and architectural consistency. While contemporary industry discourse frequently highlights distributed NoSQL key-value stores and document databases, relational systems such as MySQL (InnoDB), PostgreSQL, and Oracle remain the core transactional backbone for financial settlement platforms, enterprise identity providers, and mission-critical business workflows.

At the heart of the relational architecture lies a strict separation between logical schema definitions and physical storage mechanics. Logically, data is structured as two-dimensional relations (tables) consisting of defined attributes (columns) and tuples (rows). Physically, database engines organize these records across storage blocks, memory buffers, and tree-structured indexes.

The primary mechanism that bridges logical intent with physical correctness is the relational constraint framework. Without constraints, databases degrade into disorganized data heaps plagued by orphan records, dangling pointers, inconsistent update anomalies, and corrupt business states. Relational constraints—predominantly Candidate Keys, Primary Keys, Unique Constraints, and Foreign Key relationships—provide hard, system-level guarantees that the database engine enforces on every single transactional mutation.

This research paper provides an exhaustive, practice-oriented engineering examination of relational database integrity, key taxonomy, physical storage indexing, and mutation mechanics. Derived from production database analysis and foundational architectural principles, this paper explicitly eliminates abstract mathematical formalism in favor of practical systems engineering, physical memory and disk layout mechanics, concrete Structured Query Language (SQL) Data Definition Language (DDL) and Data Manipulation Language (DML) workflows, and real-world enterprise case studies.

2 Taxonomy and Structural Roles of Relational Keys

In a relational database, a key is an attribute or combination of attributes that serves to uniquely identify, order, or interlink tuples across relations. Understanding the precise architectural boundaries between different key classifications is essential for robust database modeling.

2.1 Candidate Keys

A Candidate Key is any minimal set of attributes within a relation that possesses the uniqueness property: no two distinct tuples in the relation can have identical values across these attributes. A relation may possess multiple candidate keys simultaneously.

For instance, consider a modern university or academy student entity containing the following attributes:

- id: A system-generated auto-incrementing integer.
- email: The official institutional email address (e.g., student@academy.edu).
- phone_number: A unique mobile contact number.
- national_id (e.g., Aadhar, Social Security Number): A government-issued civilian identification number.

Each of these four individual attributes qualifies as a Candidate Key because each one independently guarantees tuple uniqueness. If an engineer searches the relation using any of these columns, at most one single student tuple will be returned.

2.2 The Primary Key: The Chosen Relation Anchor

While a table may have several Candidate Keys, a relational table can possess exactly one Primary Key. The Primary Key is the single Candidate Key formally designated by the database designer to serve as the physical identity anchor for that relation.

The Primary Key imposes two fundamental system constraints:

1) Entity Uniqueness: No duplicate values are permitted under any circumstance.
2) Non-Nullability: The primary key column cannot contain null values. Every tuple must possess a known, concrete identifier.

The choice of which Candidate Key should be promoted to the Primary Key is not arbitrary; it directly governs the physical storage layout, index depth, and query throughput of the database engine, as analyzed in Section 3.

2.3 Composite Primary Keys

When no single attribute provides unique identification, or when modeling many-to-many junction tables, a Composite Key is utilized. A Composite Key is a primary key composed of two or more columns whose combined values are guaranteed to be unique across the relation.

A canonical example is an enrollment mapping table connecting students with courses:

- student_id: Foreign key pointing to the student relation.
- course_id: Foreign key pointing to the course relation.
- enrollment_date: Timestamp of registration.

In this schema, neither student_id nor course_id is individually unique, because a single student can enroll in multiple courses, and a single course contains multiple students. However, the compound pair (student_id, course_id) is unique, preventing duplicate registrations for the same course.

2.4 Alternate and Secondary Keys

All Candidate Keys that are not selected as the Primary Key are classified as Alternate Keys. In production SQL implementations, Alternate Keys are enforced through UNIQUE constraints accompanied by secondary indexes. Unlike primary keys, standard SQL permits alternate key columns to contain NULL values unless an explicit NOT NULL constraint is applied.

2.5 Foreign Keys: Cross-Table Referential Links

A Foreign Key is an attribute or collection of attributes in a child table that refers directly to the Primary Key (or a unique key) of a parent table. The Foreign Key establishes an enforceable parent-child bond, ensuring that a record in the child table cannot point to a non-existent entity in the parent table.

3 Physical Storage Mechanics and Primary Key Engineering

A frequent misconception in software engineering is treating primary key selection as a purely logical exercise. In high-performance relational engines like MySQL (InnoDB), the primary key directly dictates the physical organization of data on disk through Clustered Indexing.

TABLE 1
Comprehensive Comparison of Relational Key Classifications

Key Type	Multiplicity	Nullability	Core Purpose	Production Example
Candidate Key	Multiple permitted per table	Permitted unless constrained	Defines any minimal unique attribute set capable of row identification	email, phone_number, national_id
Primary Key (PK)	Exactly one per table	Strictly disallowed (NOT NULL)	Physical clustered index anchor, primary entity identifier	id INT AUTO_INCREMENT PRIMARY KEY
Composite Key	One composite PK per table	Strictly disallowed on key columns	Ensures uniqueness across multi-attribute combinations (junction tables)	PRIMARY KEY (student_id, batch_id)
Alternate Key	Multiple permitted per table	Allowed (unless marked NOT NULL)	Secondary candidate keys requiring uniqueness enforcement	email VARCHAR(255) UNIQUE
Foreign Key (FK)	Multiple permitted per table	Permitted if optional (NULL allowed)	Establishes referential integrity to a parent relation	batch_id INT REFERENCES batches(b_id)

3.1 The Clustered Index and B+ Tree Organization

In MySQL InnoDB, table data is not stored as an unordered flat file. Instead, the entire physical table is a B+ Tree structured index, termed the Clustered Index. The leaf pages of the Clustered Index do not contain pointers to data; they contain the actual row payload (all column values for that tuple).

The governing rules of InnoDB clustered indexing are:

1) The physical arrangement of rows on disk precisely mirrors the ascending sorted order of the Primary Key.
2) By default, MySQL orders result sets based on the primary key index unless an explicit ORDER BY clause overrides it.
3) Secondary indexes (non-primary indexes) do not store physical disk offsets; instead, each secondary index leaf node stores the Primary Key value of the corresponding row. Consequently, any query utilizing a secondary index performs a two-stage lookup: first searching the secondary index B+ Tree to find the Primary Key, and then traversing the Primary Key Clustered B+ Tree to retrieve the full row payload.

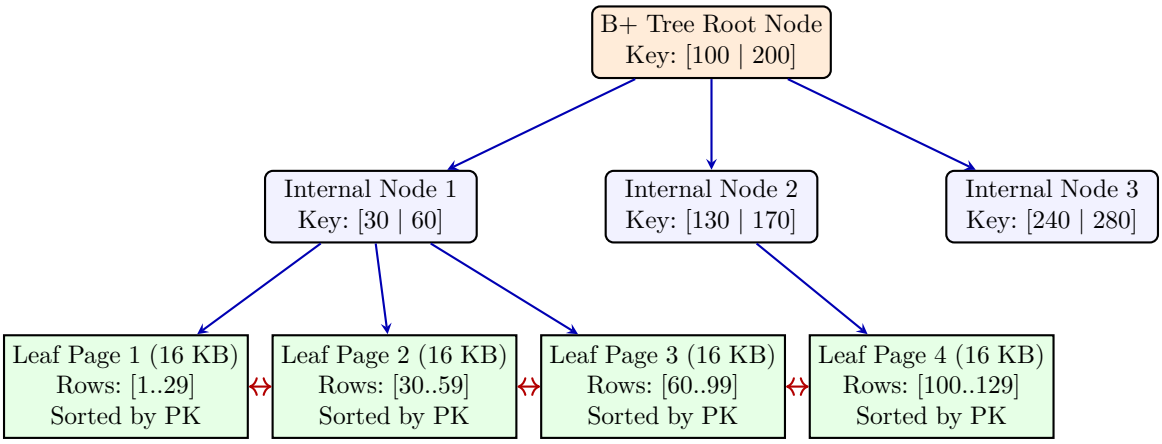


Fig. 1. Architectural layout of an InnoDB Clustered Index B+ Tree. The table rows reside directly within leaf pages ordered strictly by Primary Key. Leaf pages are doubly linked for sequential range scans.

3.2 Engineering Criteria for Optimal Primary Key Selection

Because the Primary Key forms the physical backbone of the database, choosing the correct attribute requires balancing four core engineering metrics:

3.2.1 1. Storage Footprint and Page Fan-Out

InnoDB manages storage in fixed-size pages of 16 Kilobytes (16,384 bytes). The number of index entries that can fit inside an internal B+ Tree page is known as the index fan-out.

- A standard 4-byte signed integer (INT) provides over 2.14 billion unique values. An 8-byte integer (BIGINT) provides over 9 quintillion values.

- A 12-digit string or 64-byte national identity string (such as an Aadhar number or UUID representation) consumes 16 to 64 bytes per entry.

When a primary key consumes 64 bytes compared to 4 bytes, the page fan-out drops dramatically. An internal page that could have held thousands of 4-byte key pointers can now hold only a fraction of that number. As fan-out decreases:

- The total tree height increases, requiring additional disk page reads for every point lookup.
- Every secondary index in the table balloons in size, because every secondary index record must embed a copy of the primary key.
- The database buffer pool in system memory becomes saturated with large index pages, forcing cache evictions and degrading overall system throughput.

3.2.2 2. Sorting and Comparison Performance

In B+ Tree operations (searches, node splits, binary searches within pages), key comparisons occur millions of times per second.

- Numeric Keys: Integer comparisons are executed in a single CPU cycle via hardware register instructions.
- String Keys: Lexicographical string comparisons require multi-byte memory traversals, collation rule evaluations, and variable-length decoding, resulting in significantly higher CPU overhead.

Consequently, sorting on numeric primary keys is orders of magnitude faster than sorting on textual keys.

3.2.3 3. Strict Immutability

A good Primary Key must never change. If a primary key value is modified via an UPDATE statement:

1) The database engine cannot simply modify the value in-place. Because rows are physically ordered by primary key, changing the key forces the engine to delete the row from its current physical page and insert it into a completely different page.
2) This row relocation frequently triggers costly B+ Tree page splits and rebalancing operations.
3) Every secondary index in the table must be updated to replace the old primary key pointer with the new primary key value.
4) If foreign keys in child tables reference this primary key, cascading update locks must ripple through the entire relational dependency graph, holding table and row locks that stall concurrent transactions.

3.2.4 4. Surrogate Keys vs. Natural Keys

A Natural Key is an attribute that exists organically in the real world (e.g., email address, phone number, national identity number). A Surrogate Key is a synthetic, system-generated identifier with no business meaning (e.g., an auto-incrementing integer or sequential snowflake identifier).

Real-world attributes make exceptionally poor primary keys:

- Phone numbers change when users switch cellular carriers or travel internationally.
- Email addresses change due to rebranding, personal preference, or typos during user registration.
- National IDs (Aadhar / SSN) carry severe regulatory, data privacy, and encryption compliance requirements (e.g., GDPR, HIPAA, PCI-DSS). Storing national identification strings across every secondary index and foreign key column creates severe compliance hazards.

Therefore, production systems universally adopt surrogate integer keys as Primary Keys, while enforcing natural key uniqueness through alternate unique constraints.

TABLE 2
Primary Key Selection Evaluation Matrix

Attribute	Size	Sort Speed	Immutability	Engineering Verdict
id (INT / BIGINT)	4 / 8 Bytes	Single-cycle numeric	100% Static	Optimal: Maximum fan-out, compact secondary indexes.
phone_number	10–15 Bytes	Moderate	Low (User changes)	Poor: Subject to mutation and formatting shifts.
email	30–64 Bytes	Slow (Lexicographic)	Low (User updates)	Unsuitable: High storage overhead, inflates all secondary indexes.
national_id (Aadhar)	12–64 Bytes	Slow (String checks)	Moderate	Hazardous: Privacy risks, regulatory compliance overhead.

4 Foreign Keys and Referential Integrity Foundations

The fundamental power of the relational model is its ability to establish relationships between independent entities without data duplication. This is achieved through Foreign Keys.

4.1 The Parent-Child Relational Bond

When two tables are related, they assume a hierarchical Parent-Child dynamic:

- Parent Table: The relation holding the authoritative entity and master key (typically its primary key).
- Child Table: The relation containing the referencing foreign key attribute pointing back to the parent.

Consider an educational academy system comprising batches and students:

- batches is the Parent Table, containing batch identifier b_id (Primary Key), batch_name, and instructor_name.
- students is the Child Table, containing student identifier s_id (Primary Key), student_name, psp (Problem Solving Percentage), and batch_id (Foreign Key referencing batches.b_id).

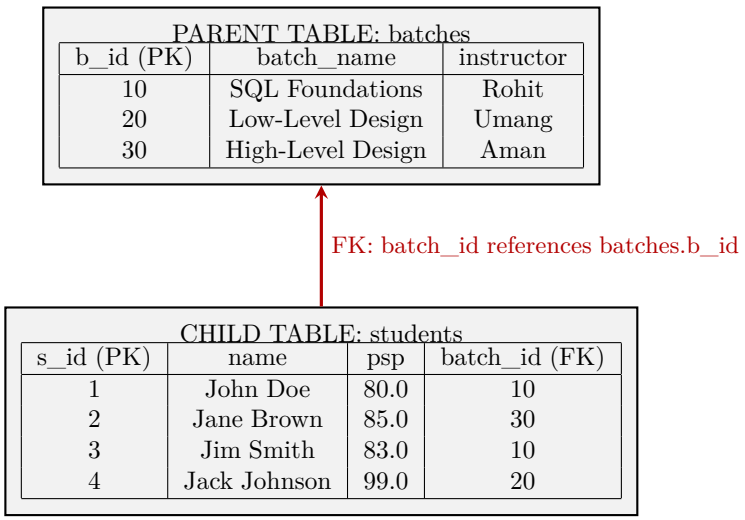


Fig. 2. The Parent-Child Foreign Key relationship between batches and students. The child table references the primary key of the parent table to guarantee referential validity.

4.2 The Danger of Orphan Records and Dangling References

What happens if the database engine does not enforce foreign key constraints? Suppose an administrator executes:

DELETE FROM batches WHERE b_id = 20;

If the relational bond is not enforced at the storage engine level:

1) The batch record with b_id = 20 is deleted from disk.
2) The student record for "Jack Johnson" (s_id = 4) still holds batch_id = 20.
3) Student 4 has now become an Orphan Record. The value 20 in batch_id is a Dangling Pointer referencing an entity that no longer exists in reality.

Orphan records cause immediate downstream systemic failure:

- Application Crashes: Application servers attempting to load batch details for Student 4 execute SQL inner joins that silently drop the student from reports, or trigger null pointer exceptions in backend microservices.
- Data Inconsistency: Automated billing engines continue charging students for batches that were decommissioned.
- Corrupt Analytical Aggregations: Analytical data pipelines fail to balance foreign key dimensions, halting business intelligence reporting.

4.3 Engine-Level Referential Constraint Interception

To eliminate orphan records, relational database engines enforce Referential Integrity. Referential integrity dictates that any foreign key value present in a child relation must match an existing primary key value in the parent relation, or be set explicitly to NULL (if nullability is permitted).

The RDBMS intercepts every data mutation before committing it to disk:

- Child Insertion Validation: If an application attempts to insert a student with batch_id = 50, but the batches table only contains identifiers 10, 20, and 30, the engine aborts the transaction immediately with a foreign key violation error (e.g., MySQL Error 1452: Cannot add or update a child row: a foreign key constraint fails).
- Parent Modification Interception: If an application attempts to delete a parent batch or change its primary key while child student rows point to it, the engine enforces configured referential action rules.

4.4 DDL Creation and Destruction Dependencies

Referential constraints introduce a strict directional dependency in SQL Data Definition Language (DDL) execution:

1) Creation Order: The Parent table must be created before the Child table. A table definition that declares a foreign key referencing a non-existent parent table will fail compilation immediately.
2) Destruction Order: The Child table must be dropped before the Parent table can be deleted. Attempting to execute DROP TABLE batches; while students exists will result in MySQL Error 1217: Cannot delete or update a parent table: a foreign key constraint fails.

5 Mutation Dynamics: Operational Semantics of ON UPDATE and ON DELETE

When a record in a parent table undergoes mutation—specifically when a parent row is deleted or its primary key value is updated—the database engine must determine how to handle dependent child rows. SQL provides four standardized behavioral policies defined via the ON UPDATE and ON DELETE clauses.

5.1 RESTRICT / NO ACTION (The Defensive Default)

In standard MySQL InnoDB, if no explicit rule is specified, the engine applies RESTRICT (which behaves identically to NO ACTION in standard transaction contexts).

- Behavior: If any child row references the parent record, the database engine prohibits any modification or deletion of that parent record. The transaction is instantly aborted and rolled back.
- Operational Workflow: To delete batch 20, an administrator must first manually reassign or delete all students belonging to batch 20. Only after zero child records reference batch 20 will the engine permit the deletion of batch 20.
- Use Case: Mission-critical financial ledgers and core entity master records where silent automatic modifications would introduce operational ambiguity.

5.2 CASCADE (Propagative Synchronization)

The CASCADE action instructs the database engine to automatically propagate parent mutations down to all dependent child records.

- ON UPDATE CASCADE: If parent batch 10 has its identifier updated to 15 (UPDATE batches SET b_id = 15 WHERE b_id = 10), the database engine automatically updates the batch_id column in all student rows referencing 10 to 15. Referential integrity is preserved automatically without human intervention.
- ON DELETE CASCADE: If parent batch 10 is deleted (DELETE FROM batches WHERE b_id = 10), the engine automatically executes deletions for every single student enrolled in batch 10.
- Production Risk: While ON UPDATE CASCADE is generally safe and beneficial, ON DELETE CASCADE is hazardous. Deleting a parent category or batch could unintentionally wipe out thousands of dependent student records.

5.3 SET NULL (Decoupled Disassociation)

The SET NULL action breaks the relational bond upon parent modification without destroying the child record.

- Behavior: When the referenced parent record is deleted or updated, the foreign key column in all dependent child rows is automatically set to NULL.
- Prerequisite: The foreign key column in the child table must be nullable (it cannot have the NOT NULL constraint).
- Operational Meaning: In our academy system, if batch 20 is deleted, student 4 (Jack Johnson) is not deleted from the database. Instead, Jack's batch_id becomes NULL. Jack remains an active student in the academy system, classified as unassigned and ready to be enrolled into a future batch.
- Use Case: Optional relationships where the child entity has an independent existence outside the lifespan of the parent entity.

TABLE 3
Comparative Matrix of Referential Action Rules

Action Rule	Parent Deletion Behavior	Parent Key Update Behavior	Child Data Retention	Primary Risk Profile
RESTRICT / NO ACTION	Rejects deletion if child rows exist (Error 1451).	Rejects update if child rows exist (Error 1451).	100% preserved; blocks unsafe operations.	Application errors if deletions are not preceded by manual cleanup.
CASCADE	Automatically deletes all referencing child rows.	Automatically updates foreign keys in child rows.	Child rows deleted on parent drop; child keys updated on parent update.	Catastrophic mass data deletion if parent is inadvertently dropped.
SET NULL	Child foreign key columns set to NULL.	Child foreign key columns set to NULL.	100% retained; records disassociated from parent.	Requires nullable columns; queries must handle null value checks.

5.4 SET DEFAULT

Under SET DEFAULT, when a parent record is mutated, the child table foreign key column is automatically assigned a pre-configured default value (e.g., reassigning unbatched students to a designated Holding Batch with b_id = 1). While supported in standard SQL and PostgreSQL, MySQL InnoDB does not support SET DEFAULT foreign key actions.

6 Enterprise System Architectural Case Studies

To demonstrate how key selection and referential actions govern enterprise system design, we examine two real-world architectural patterns reflecting the concepts analyzed in production environments.

6.1 Case Study A: Unified Identity Provider Lifecycle (The Google Account SSO Model)

Consider the multi-service architecture of a global identity provider such as Google Single Sign-On (SSO):

- Parent Entity: The central Google User Account (user_id, email, status).
- Child Entities: Downstream service workspaces:
  - gmail_inbox (inbox_id, user_id, storage_used)
  - drive_files (file_id, user_id, file_path)
  - photos_albums (album_id, user_id, photo_count)
  - youtube_channel (channel_id, user_id, subscriber_count)

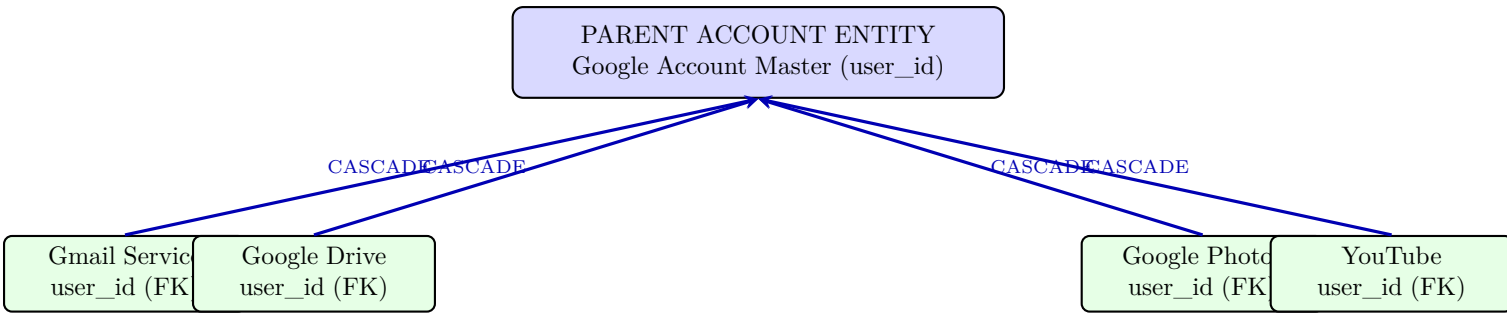


Fig. 3. The Unified Identity lifecycle architecture. A deletion of the central Google Account cascades across all downstream services, guaranteeing complete personal data purging in compliance with data privacy mandates.

6.1.1 Architectural Policy Analysis

In this unified identity architecture, the appropriate referential policies are:

- ON UPDATE CASCADE: If the account's internal routing identifier changes, that change is immediately synchronized across Drive, Photos, and Gmail tables.
- ON DELETE CASCADE: When a user requests permanent account termination under data privacy legislation (e.g., GDPR Right to be Forgotten), the database must guarantee complete eradication of all associated files, mailboxes, and photos. Leaving orphan records in Drive or Photos after the central account is deleted would represent a major regulatory violation and security flaw.

6.2 Case Study B: E-Commerce Audit Trails and Settlement (The Amazon Order Model)

Now contrast the identity lifecycle with an e-commerce order processing platform such as Amazon:

- Parent Entities:
  1) customers (customer_id, name, shipping_address)
  2) saved_payment_methods (card_id, customer_id, tokenized_card_number)
- Child Entities:
  1) orders (order_id, customer_id, payment_card_id, total_amount, tax_paid)
  2) order_items (item_id, order_id, sku, price)

6.2.1 Architectural Policy Analysis

What occurs when a customer deletes an expired credit card from their profile, or even closes their retail purchasing account?

- Catastrophic Failure of CASCADE: If orders had an ON DELETE CASCADE relationship with saved_payment_methods, deleting an expired Visa card would trigger the database engine to delete all historical orders ever placed with that card! This would destroy taxable transaction logs, eliminate fulfillment records, and wipe out company revenue history.
- The SET NULL / Immutability Architecture: To ensure financial compliance, historical orders must never be deleted. The foreign key on payment_card_id is configured with ON DELETE SET NULL. If the customer deletes their saved credit card, the historical order retains all fulfillment data, while the payment reference safely transitions to null (with financial audit tokens preserved in an immutable transaction ledger). Furthermore, customer account deactivations use Soft Deletion flags (is_active = false) rather than physical deletions, preserving complete referential integrity across fiscal audit cycles.

7 Production SQL Implementation Protocols

This section provides complete, syntactically validated SQL scripts demonstrating database lifecycle management, strict referential constraint definitions, and transactional validation using the Academy domain model analyzed in Section 4.

7.1 Database Creation and Context Selection

Every schema definition begins by provisioning an isolated database namespace and establishing execution context:

```sql
-- 1. Create the dedicated database namespace
CREATE DATABASE IF NOT EXISTS academy_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- 2. Switch session context to the new database
USE academy_db;
```

7.2 Parent Table Definition: batches

The parent table must be provisioned first. It establishes the primary key anchor:

```sql
-- Create the parent table
CREATE TABLE IF NOT EXISTS batches (
    b_id INT AUTO_INCREMENT PRIMARY KEY,
    batch_name VARCHAR(100) NOT NULL,
    instructor_name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

7.3 Child Table Definition with Referential Actions: students

The child table defines the foreign key constraint, enforcing both ON UPDATE CASCADE and ON DELETE SET NULL:

```sql
-- Create the child table referencing batches
CREATE TABLE IF NOT EXISTS students (
    s_id INT AUTO_INCREMENT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    psp DOUBLE NOT NULL,
    batch_id INT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    -- Explicit Foreign Key Constraint
    CONSTRAINT fk_students_batches
        FOREIGN KEY (batch_id)
        REFERENCES batches(b_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
) ENGINE=InnoDB;
```

7.4 Data Manipulation: Populating Master and Dependent Data

Data insertion must adhere to dependency order: parent records first, followed by child records referencing valid parent identifiers:

```
-- 1. Seed Parent Batches
INSERT INTO batches (b_id, batch_name, instructor_name)
VALUES
    (10, 'SQL & Relational Databases', 'Rohit'),
    (20, 'Low-Level Design Masterclass', 'Umang'),
    (30, 'High-Level System Design', 'Aman');

-- 2. Seed Dependent Student Tuples
INSERT INTO students (s_id, student_name, email, psp, batch_id)
VALUES
    (1, 'John Doe', 'john@academy.edu', 80.0, 10),
    (2, 'Jane Brown', 'jane@academy.edu', 85.0, 30),
    (3, 'Jim Smith', 'jim@academy.edu', 83.0, 10),
    (4, 'Jack Johnson', 'jack@academy.edu', 99.0, 20),
    (5, 'Jenny Doe', 'jenny@academy.edu', 92.5, 20);
```

7.5 Data Retrieval and Relational Projections

Standard queries project attributes across the referential boundary using structured joins and conditional filters:

```
-- Project students with their corresponding batch details
SELECT
    s.s_id,
    s.student_name,
    s.psp,
    b.batch_name,
    b.instructor_name
FROM students s
LEFT JOIN batches b ON s.batch_id = b.b_id
ORDER BY s.s_id ASC;
```

7.6 Verification of Constraint Enforcement

7.6.1 Test 1: Child Insertion with Invalid Foreign Key

Attempting to insert a student pointing to non-existent batch 99:

```
-- This statement will fail with MySQL Error 1452
INSERT INTO students (student_name, email, psp, batch_id)
VALUES ('Orphan Candidate', 'orphan@academy.edu', 75.0, 99);
```

The engine halts execution:

ERROR 1452 (23000): Cannot add or update a child row:
a foreign key constraint fails (‘academy_db‘.‘students‘,
CONSTRAINT ‘fk_students_batches‘ FOREIGN KEY (‘batch_id‘)
REFERENCES ‘batches‘ (‘b_id‘) ON UPDATE CASCADE ON DELETE SET NULL)

7.6.2 Test 2: Verification of ON UPDATE CASCADE

Updating batch 10 to identifier 15 in the parent table:

```
UPDATE batches SET b_id = 15 WHERE b_id = 10;
```

Querying the child table reveals that students 1 and 3 (previously associated with batch 10) have automatically had their batch_id updated to 15 without issuing an explicit update against students:

```
+------+------------+------+----------+
| s_id | name       | psp  | batch_id |
+------+------------+------+----------+
|    1 | John Doe   | 80.0 |       15 |
|    3 | Jim Smith  | 83.0 |       15 |
+------+------------+------+----------+
```

7.6.3 Test 3: Verification of ON DELETE SET NULL

Deleting batch 20 from the parent table:

```
DELETE FROM batches WHERE b_id = 20;
```

Querying students 4 and 5 (formerly in batch 20) reveals that their records remain intact, but their batch_id attribute has transitioned cleanly to NULL:

```
+------+--------------+------+----------+
| s_id | name         | psp  | batch_id |
+------+--------------+------+----------+
|    4 | Jack Johnson | 99.0 |     NULL |
|    5 | Jenny Doe    | 92.5 |     NULL |
+------+--------------+------+----------+
```

Referential integrity is maintained, no orphan records exist, and zero application data was lost.

8 The Sakila Reference Database Architecture

To observe relational integrity across industrial-scale schemas, database engineers frequently reference the canonical Sakila database. Sakila was designed by MySQL architects to model a commercial video rental enterprise, incorporating complex referential constraints, composite keys, and cascade chains.

8.1 Core Entity Relationships in Sakila

Sakila demonstrates multi-tiered foreign key dependencies across four functional operational domains:

1) Inventory and Catalog Management:
   - film relates to language via foreign key language_id.
   - film_actor acts as a many-to-many junction table between film and actor, utilizing a Composite Primary Key consisting of (actor_id, film_id).
   - inventory represents physical stock copies in stores, holding foreign keys referencing both film(film_id) and store(store_id).
2) Customer Operations and Rentals:
   - rental captures the temporal checkout event, maintaining foreign keys pointing to inventory(inventory_id), customer(customer_id), and staff(staff_id).
3) Payment Settlement:
   - payment references rental(rental_id) and customer(customer_id), ensuring that financial ledger rows cannot exist without an authorized customer and rental record.

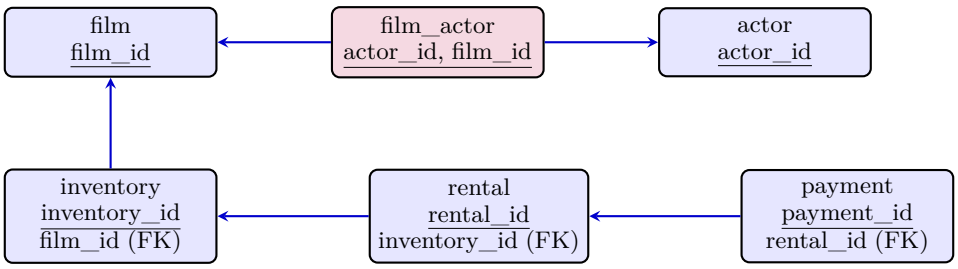


Fig. 4. Architectural dependency graph of the canonical Sakila database. Foreign key linkages bind inventory, rentals, and payments, preventing orphaned transactions across multi-tiered relational workflows.

8.2 Architectural Lessons from Sakila

The Sakila schema highlights two critical production guidelines:

- Every Foreign Key Must Be Indexed: In the child table rental, the column inventory_id is a foreign key. MySQL automatically creates an index on this foreign key column. Without this index, any deletion or update in the parent inventory table would force the database engine to perform a full table scan across millions of rental rows to verify referential integrity, resulting in severe lock contention.
- Strict Isolation of Financials: Payments in Sakila reference rentals, but historical payments are protected from cascade deletion via RESTRICT rules, preventing accidental loss of financial accounting records.

9 Production Engineering Best Practices

To conclude this architectural analysis, we outline essential engineering guidelines for database designers building high-throughput relational systems:

1) Always Use Compact Surrogate Primary Keys: Standardize on 4-byte INT or 8-byte BIGINT auto-incrementing identifiers for primary keys. Never use national identification numbers, phone numbers, or unhashed email strings as primary keys.
2) Index Every Foreign Key Column: Database engines automatically create indexes on Primary Keys, but they do not always automatically index Foreign Key columns in child tables. Always verify that an index exists on every foreign key column to prevent full table scans and row locks during parent table mutations.
3) Explicitly Declare Referential Rules: Never leave foreign key actions to implicit defaults. Always explicitly define ON UPDATE CASCADE and either ON DELETE RESTRICT or ON DELETE SET NULL to make business intent unmistakable in schema DDL.
4) Avoid Deep Cascade Trees: Chaining CASCADE rules across five or six hierarchical tiers (e.g., Company to Department to Team to Project to Task to Comment) creates massive transactional lock cascades. A single deletion at the root can lock hundreds of thousands of rows, leading to transaction timeouts and deadlocks. In deeply nested architectures, utilize asynchronous soft deletion workers instead of synchronous deep database cascades.
5) Enforce Alternate Key Uniqueness Independently: Protect natural keys (such as email and phone numbers) using explicit UNIQUE constraints rather than overloading them as primary keys.

10 Conclusion

Relational database integrity is the bedrock of dependable software engineering. By understanding the physical reality of primary key clustered indexes, the storage footprint overhead of candidate keys, and the operational mechanics of foreign key referential constraints, software engineers can design resilient, high-throughput architectures that completely eliminate orphan records and update anomalies.

This paper has demonstrated the complete taxonomy of relational keys, proved why compact immutable surrogate integers outperform natural strings as primary keys, detailed the exact operational mechanics of ON UPDATE and ON DELETE rules (CASCADE, SET NULL, RESTRICT), analyzed production case studies from Google SSO and Amazon Order management, and provided validated SQL implementation protocols. Adherence to these engineering principles ensures that relational systems remain both performant under massive scale and structurally immutable against data corruption.

References

[1] E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.
[2] C. J. Date, An Introduction to Database Systems, 8th ed. Boston: Addison-Wesley, 2004.
[3] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York: McGraw-Hill, 2020.
[4] Oracle Corporation, “MySQL 8.4 Reference Manual: InnoDB Storage Engine and Clustered Indexes,” Oracle Technology Network, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/innodb-index-types.html
[5] Oracle Corporation, “Sakila Sample Database Architecture and Schema Documentation,” Oracle Corporation, 2024. [Online]. Available: https://dev.mysql.com/doc/sakila/en/